

ASCII Studio Project Documentation

1. Project Overview

Vision & Mission

ASCII Studio aims to provide a user-friendly and versatile web application for converting text into customizable ASCII art. The mission is to empower users, from developers to casual enthusiasts, to effortlessly generate, personalize, and deploy ASCII art for various creative and practical applications.

Core Objectives

- To develop a robust backend in Go for efficient ASCII art generation and customization.
- To create an intuitive and responsive web interface using HTML, CSS, and JavaScript for seamless user interaction.
- To enable comprehensive customization options, including banner styles, colors, and text alignment.
- To facilitate easy deployment and portability through Docker containerization.
- To allow users to save and manage their ASCII art projects locally.

Target Audience

The primary target audience includes developers, designers, content creators, and individuals seeking unique textual art for social media, presentations, or personal projects. The Docker deployment option also caters to users who prefer self-hosting and containerized applications.

2. Product Requirements Document (PRD)

Functional Requirements

- **Text to ASCII Art Conversion:** The application shall convert user-inputted text into ASCII art using various banner styles.
- **Banner Style Selection:** Users shall be able to select from predefined banner styles (e.g., standard, shadow, thinkertoy) .
- **Color Application:** Users shall be able to apply colors to individual characters or entire sections of the generated ASCII art.

- **Text Alignment:** The application shall support left, right, center, and justify text alignment options for the ASCII art output.
- **Live Preview:** A live preview feature shall display the ASCII art as it is being generated and customized.
- **Download Output:** Users shall be able to download the generated ASCII art as a plain `.txt` file.
- **Responsive Web Interface:** The user interface shall be responsive and accessible across various devices and screen sizes.
- **Docker Deployment:** The application shall be deployable as a Docker container.

Non-Functional Requirements

- **Performance:** ASCII art generation and preview should be near-instantaneous for typical text inputs.
- **Usability:** The interface should be intuitive and easy to navigate for all user levels.
- **Reliability:** The application should handle various inputs gracefully and maintain stability.
- **Security:** banner files should be processed securely to prevent malicious code execution.
- **Maintainability:** The codebase should be well-structured, documented, and easy to maintain and extend.
- **Scalability:** The backend should be designed to handle a growing number of requests, though initial focus is on single-user instances.

User Stories

- As a user, I want to type text and see it instantly converted to ASCII art so I can quickly visualize my ideas.
- As a user, I want to choose different banner styles to give my ASCII art a unique look and feel.
- As a user, I want to apply specific colors to parts of my ASCII art to highlight certain words or phrases.
- As a user, I want to align my ASCII art to fit different contexts, such as a terminal window or a document.
- As a user, I want to download my ASCII art as a text file so I can use it anywhere.
- As a developer, I want to deploy ASCII Studio easily using Docker so I can self-host it.

UI/UX Principles

- **Simplicity:** The interface should be clean and uncluttered, focusing on core functionality.
- **Instant Feedback:** Actions should result in immediate visual feedback (e.g., live preview).
- **Consistency:** UI elements and interactions should be consistent throughout the application.
- **Accessibility:** The application should be usable by individuals with diverse needs, adhering to web accessibility standards where possible.

3. Technical Specification

Tech Stack Overview

- **Backend:** Go (for high performance and concurrency)
- **Frontend:** HTML, CSS, JavaScript (for a responsive and interactive web interface)
- **Containerization:** Docker (for easy deployment and environment consistency)

Backend Architecture

The Go backend will be responsible for handling all core logic related to ASCII art generation, color application, and alignment. It will expose a set of APIs for the frontend to consume.

ASCII Conversion Engine (`internal/ascii`)

- This module will contain the logic for converting input text into ASCII art using various banner files.
- It will load and parse banner files (e.g., `.flf` format or custom formats).
- Functions for applying different banner styles will reside here.

Color Engine (`internal/color`)

- This module will manage the application of colors to the ASCII art.
- It will support coloring individual characters, words, or the entire output.
- Color parsing and application logic will be implemented here.

Alignment Engine (`internal/alignment`)

- This module will handle the alignment of the generated ASCII art.
- It will provide functions for left, right, center, and justified alignment.

Storage Module (`internal/storage`)

- This module will manage the local saving and loading of user projects.
- It will interact with the local filesystem or a simple database (e.g., SQLite) for project persistence.

Renderer Module (`internal/renderer`)

- This module will be responsible for combining the output from the ASCII, Color, and Alignment engines to produce the final ASCII art string.
- It will handle the rendering process before sending the result to the frontend.

Frontend Architecture

The frontend will be a single-page application (SPA) providing the user interface and interacting with the Go backend via RESTful APIs.

Live Preview Logic

- JavaScript will be used to capture user input (text, banner selection, colors, alignment).
- API calls will be made to the backend to generate the ASCII art.
- The generated ASCII art will be displayed in real-time within a designated preview area.

State Management

- Frontend state (current text, selected banner, colors, alignment, saved projects) will be managed using plain JavaScript or a lightweight library.

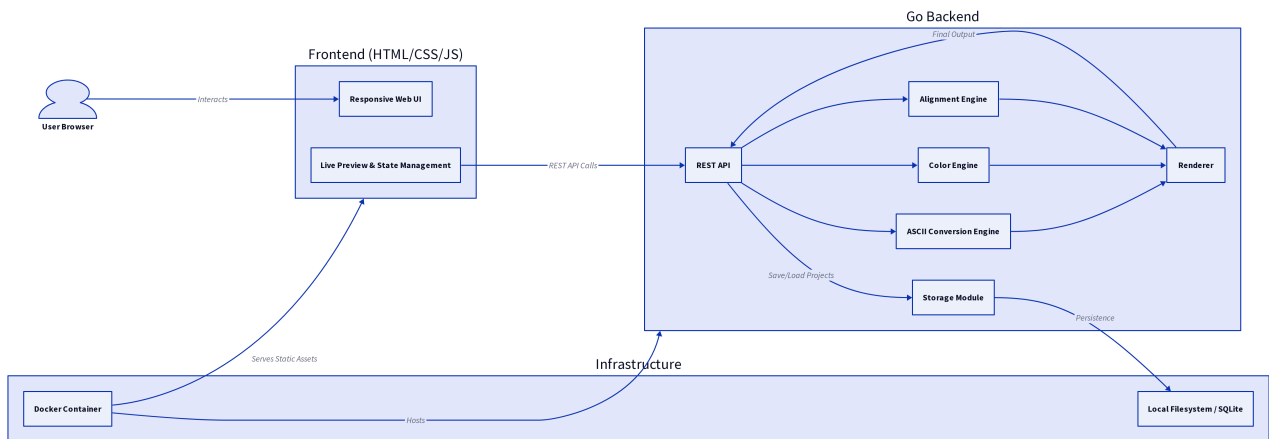
API Definitions

The backend will expose RESTful API endpoints. Examples include:

- `POST /api/generate` : Accepts text, banner style, color options, and alignment, returns generated ASCII art.
- `POST /api/save-project` : Accepts project data, saves it locally.
- `GET /api/load-project/{id}` : Retrieves a saved project.
- `GET /api/banners` : Returns a list of available banner styles.

4. System Architecture

Architecture Overview



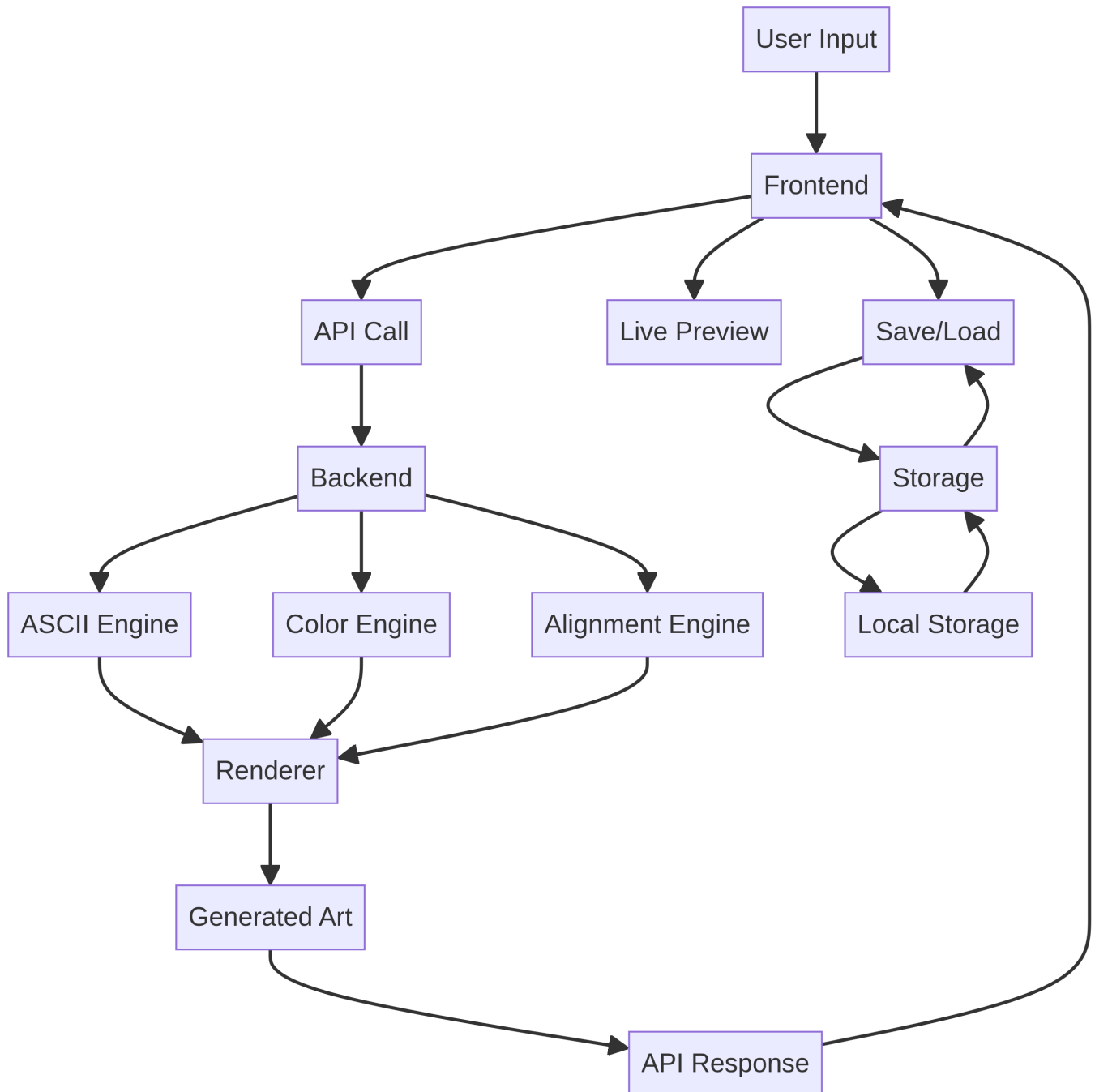
Directory Structure

Plain Text

```
ascii-studio/
├── cmd/
│   └── server/                # Main application entry point
├── internal/
│   ├── ascii/                # ASCII conversion logic
│   ├── color/                # Color application logic
│   ├── alignment/            # Text alignment logic
│   ├── storage/              # Project storage logic
│   └── renderer/             # Final ASCII art rendering
├── web/
│   ├── static/
│   │   ├── css/              # Frontend CSS files
│   │   ├── js/               # Frontend JavaScript files
│   │   └── images/            # Frontend image assets
│   └── templates/            # HTML templates for the web interface
├── banners/                  # Directory for banner files (standard and custom)
└── Dockerfile                 # Docker build instructions
```

```
├─ docker-compose.yml    # Docker Compose configuration for multi-service de
└─ README.md             # Project README file
```

Data Flow Diagram



Component Interaction

- The **Frontend** (HTML, CSS, JavaScript) serves as the user interface, capturing input and displaying output.
- The **Go Backend** processes requests from the frontend, orchestrates the ASCII, Color, and Alignment engines, and manages project storage.

- The **ASCII Conversion Engine** uses banner files to transform text.
- The **Color Engine** and **Alignment Engine** modify the output from the ASCII engine.
- The **Renderer** combines these modifications into the final ASCII art.
- The **Storage Module** handles persistence of user projects.

5. Deployment & DevOps

Dockerization Strategy

- A `Dockerfile` will be provided to build a lightweight Docker image for the Go backend and serve the static frontend assets.
- `docker-compose.yml` will facilitate easy local deployment, potentially including a simple database if the storage module evolves beyond filesystem-based storage.

CI/CD Considerations

- Automated testing (unit, integration) will be implemented for the Go backend.
- CI/CD pipelines can be set up to automatically build Docker images and deploy updates to a chosen hosting environment upon code commits.

6. Development Roadmap

Phase 1: MVP (Core ASCII & Web UI)

- **Goal:** Establish the foundational text-to-ASCII conversion with a basic web interface.
- **Features:** Input text, select standard banner styles, live preview, download as TXT.
- **Tech Focus:** Go backend for ASCII conversion, basic HTML/CSS/JS frontend.

Phase 2: Enhanced Features (Colors, Alignment, Custom Banners)

- **Goal:** Introduce customization options to enrich the ASCII art generation.
- **Features:** Color engine, alignment engine, upload custom banner files.
- **Tech Focus:** Extend Go backend with color and alignment logic, enhance frontend UI for new options.

Phase 3: Persistence & Advanced Export

- **Goal:** Enable users to manage their projects and offer more export formats.

- **Features:** Local project saving and loading, ASCII editor for post-generation edits, export as HTML.
- **Tech Focus:** Implement storage module, integrate an in-browser text editor.

Phase 4: Optimization & Deployment

- **Goal:** Refine performance, ensure robustness, and simplify deployment.
- **Features:** Performance optimizations, comprehensive error handling, single-command Docker deployment.
- **Tech Focus:** Profiling Go application, Docker Compose setup, security audits.

7. Risks & Mitigation

Potential Challenges

- **Performance with Large Inputs:** Generating ASCII art for very long texts or complex banners might be slow.
- **Custom Banner File Security:** Uploading arbitrary files poses a security risk.
- **Cross-Browser Compatibility:** Ensuring consistent UI/UX across different web browsers.
- **Complex Color/Alignment Logic:** Implementing advanced coloring and justification might be intricate.

Strategic Solutions

- **Performance:** Implement caching for frequently used banner files, optimize Go string manipulation, consider streaming output for very large texts.
- **Custom Banner Security:** Validate uploaded files rigorously (e.g., file type, content sanity checks), run file processing in isolated environments if possible.
- **Cross-Browser Compatibility:** Use modern web standards, test thoroughly across major browsers, consider a frontend framework if complexity increases.
- **Complex Logic:** Break down complex features into smaller, manageable sub-modules, utilize Go's concurrency features where appropriate, write extensive unit tests.